

1. Current technical status

What exists

- Existing AI/software engineering work has established components relevant to the research direction.
- An EDA-Agent research project explored multi-agent reasoning for RTL verification/debugging.
- Robotics work has included multi-robot simulation and a central-intelligence architecture concept.
- Current Activate proposal is the next research step: an integrated world-model + synthesis + planning system.

What is NOT yet proven

- No physical robot demonstration of the full proposed architecture yet.
- No claim of AGI or state-of-the-art superiority at this stage.
- Quantitative comparison against matched model-free/model-based baselines is a planned experiment.
- The core scientific question remains open and falsifiable.

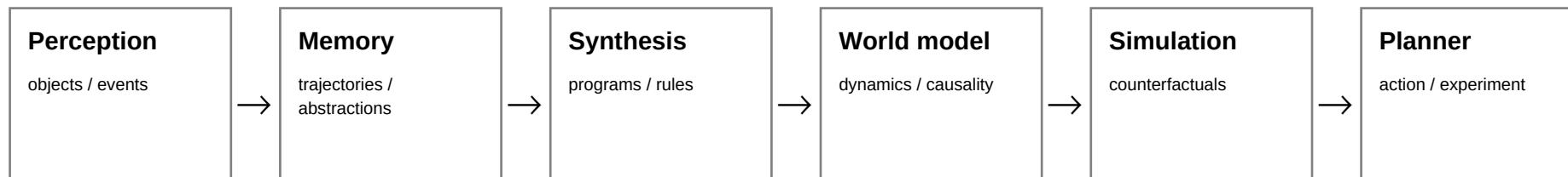
Immediate technical target

Build a reproducible simulated environment where an agent must infer hidden dynamics from limited observations, synthesize a compact executable model, actively test hypotheses, plan counterfactuals, and transfer the learned abstraction to a changed environment.

Research status: concept → integrated software prototype → simulation validation → physical validation

Evidence will be reported using held-out environments, ablations, interaction-sample counts, prediction error, transfer success, and compute/time.

2. Proposed technical architecture and evaluation



Core experiment

Unseen environment → limited observations → competing hypotheses → informative actions → model update → predicted outcome → transfer to held-out configuration.

Metrics

Sample efficiency; dynamics prediction error; task success; transfer success; planning latency; compute per solved task; robustness under changed dynamics.

Baselines + ablations

Baselines: rule-based planner, model-free RL, model-based RL. Ablate: program synthesis, explicit memory, counterfactual simulation, active experimentation. The purpose is to determine whether the proposed components provide measurable benefit rather than assuming they do.

3. First product hypothesis and business learning

First product

Adaptive autonomy software layer / SDK for robotics developers. Initial use case: environments where task-specific data collection and engineering are expensive because objects, layouts, or operating conditions change.

Customer hypothesis

Robotics OEMs, industrial automation companies, logistics operators, and autonomous-machine developers. Initial customer discovery should test willingness to pay for lower adaptation cost and faster deployment.

Economic measurements

Measure compute cost per solved task, interaction data required, engineering hours for adaptation, deployment latency, and failure/recovery rates. Pricing is intentionally not fixed before these measurements.

Current business-model learning

The technology is still pre-product. Existing AI/software work provides experience building and deploying technical systems, but the proposed architecture has not yet been validated commercially. The first commercial experiment is therefore a technical + customer-discovery loop: identify an expensive adaptation problem, reproduce it in simulation, quantify the baseline effort, test the proposed approach, and only then define a software licensing / usage model.

4. Two-year technical work plan

0–6 mo	Cognitive kernel Memory + symbolic reasoning + program synthesis + verifier;
6–12 mo	World-model learning State/dynamics representation; uncertainty; active experiments;
12–18 mo	Counterfactual planning Simulation/search; transfer across held-out configurations;
18–24 mo	Robotics validation ROS 2 + Gazebo integration → hardware-in-the-loop → physical
End-of-fellowship exam A previously unseen environment is provided with limited interaction data. Success requires the system to: 1. infer useful hidden rules; 2. choose experiments that reduce uncertainty; 3. construct a predictive executable model; 4. plan using counterfactual rollouts; 5. transfer an abstraction to a changed configuration; 6. report reproducible performance and compute cost. Physical deployment is gated by simulation safety tests, action limits, human override, and hardware-in-the-loop validation.	